

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Perform dry run testing to trace the execution of algorithms, analyze intermediate steps, and verify the correctness of processes. (BL3)

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2

Total Marks:20

Annexure A
DRY RUN TEST

Code of Conduct:

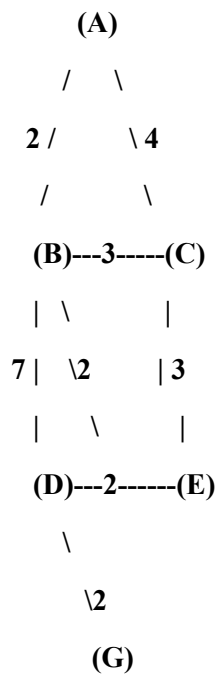
I B.Satya Hyndavi(192311178) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1: Dry Run of A Search Algorithm (10 Marks)*

Consider the following graph.



Heuristic Values

Node h(n)

A 7

B 6

C 4

D 3

E 2

G 0

Task:

Perform a dry run of the A* Search algorithm from A to G.

For each iteration, show:

- **Current node**
- **Open List**
- **Closed List**
- **g(n)**
- **h(n)**

- $f(n) = g(n) + h(n)$

Finally,

1. Identify the optimal path.
2. Calculate the total path cost.

Problem

A robot must find the shortest path from a **Start node (A)** to a **Goal node (G)** in a weighted graph. The challenge is to determine the optimal path efficiently by considering both the actual travel cost and an estimate of the remaining distance to the goal.

Solution

- **A* Search Algorithm:** A* is an informed search algorithm that selects the next node based on the evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from the start node to the current node and $h(n)$ is the heuristic estimate from the current node to the goal.
- **Path Evaluation:** At each iteration, A* maintains an **Open List** (nodes to be explored) and a **Closed List** (already explored nodes). The node with the lowest $f(n)$ value is selected for expansion.
- **Heuristic Function:** The heuristic values guide the search toward the goal while reducing unnecessary exploration. A good heuristic improves efficiency without affecting optimality.
- **Optimal Path Selection:** The algorithm updates the cost of neighboring nodes whenever a shorter path is found and continues until the goal node is reached.
- **Recommendation:** A* Search is the best approach because it combines the actual path cost with heuristic information, enabling it to find the shortest path efficiently while exploring fewer nodes than uninformed search algorithms.

Python Code

```
import heapq
```

```
graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('C', 3), ('D', 7), ('E', 2)],
    'C': [('E', 3)],
    'D': [('E', 2), ('G', 2)],
    'E': [],
    'G': []
}
```

```
heuristic = {
```

```
'A': 7,  
'B': 6,  
'C': 4,  
'D': 3,  
'E': 2,  
'G': 0  
}
```

```
def astar(start, goal):  
    open_list = [(heuristic[start], 0, start, [start])]   
    visited = set()  
  
    while open_list:  
        f, g, node, path = heapq.heappop(open_list)  
  
        if node == goal:  
            return path, g  
  
        if node in visited:  
            continue  
  
        visited.add(node)  
  
        for neighbor, cost in graph[node]:  
            if neighbor not in visited:  
                new_g = g + cost  
                new_f = new_g + heuristic[neighbor]  
                heapq.heappush(open_list,  
                               (new_f, new_g, neighbor, path + [neighbor]))  
  
    return None, float('inf')
```

```
path, cost = astar('A', 'G')
```

```
print("Optimal Path:", " -> ".join(path))
```

```
print("Total Path Cost:", cost)
```

Output

```
Optimal Path: A -> B -> D -> G
Total Path Cost: 11

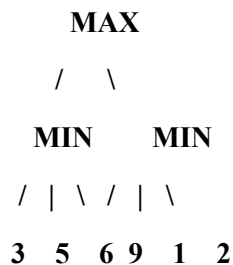
Process finished with exit code 0
```

Report

A* Search is the most suitable algorithm for pathfinding because it efficiently combines the actual path cost with heuristic estimates to guide the search toward the goal. It explores fewer nodes than uninformed search algorithms while still guaranteeing the optimal path when the heuristic is admissible. For the given graph, the optimal path is **A → B → D → G** with a **total path cost of 11**.

Question 2: Dry Run of Minimax with Alpha-Beta Pruning (10 Marks)

Consider the following game tree.



Task:

Perform a dry run of the Minimax algorithm with Alpha-Beta Pruning by evaluating the tree from left to right.

For each node, show:

- Alpha (α)
- Beta (β)
- Selected value
- Any node that gets pruned

Finally,

1. Determine the best move for MAX.
2. State the final minimax value.

3. List the pruned nodes (if any).

Problem

A game tree consists of a MAX node at the root and two MIN nodes as its children. The leaf nodes contain utility values. The objective is to perform a dry run of the Minimax algorithm with Alpha-Beta Pruning by evaluating the tree from left to right, showing the alpha (α), beta (β), selected value, and any pruned nodes. Finally, determine the best move for MAX and the final minimax value.

Solution

- Minimax Algorithm: The Minimax algorithm is used in two-player games to determine the optimal move. MAX tries to maximize the score, while MIN tries to minimize it.
- Alpha-Beta Pruning: Alpha-Beta Pruning improves the efficiency of Minimax by eliminating branches that cannot affect the final decision.
 - Alpha (α): Best value found so far for the MAX player.
 - Beta (β): Best value found so far for the MIN player.
 - If $\beta \leq \alpha$, the remaining branches are pruned because they cannot influence the final result.
- Tree Evaluation (Left to Right):

Step 1: Root (MAX)

- $\alpha = -\infty$
- $\beta = +\infty$

Evaluate the left MIN node first.

Step 2: Left MIN Node

Initial values:

- $\alpha = -\infty$
- $\beta = +\infty$

Leaf	Selected Value	α	β	Pruning
3	3	$-\infty$	3	No
5	$\min(3,5)=3$	$-\infty$	3	No
6	$\min(3,6)=3$	$-\infty$	3	No

Left MIN returns = 3

Step 3: Update Root (MAX)

Current best value:

- $\text{MAX} = \max(-\infty, 3) = 3$

Update:

- $\alpha = 3$
- $\beta = +\infty$

Now evaluate the right MIN node.

Step 4: Right MIN Node

Inherited values:

- $\alpha = 3$
- $\beta = +\infty$

Leaf	Selected Value	α	β	Pruning
9	9	3	9	No
1	$\min(9, 1) = 1$	3	1	Yes ($\beta \leq \alpha$)
2	Pruned	—	—	Yes

Since $\beta = 1 \leq \alpha = 3$, the remaining child (2) is pruned.

Right MIN returns = 1

Step 5: Final Decision at Root

MAX chooses:

- $\max(3, 1) = 3$

Therefore, MAX selects the left subtree.

Dry Run Summary

Node	Alpha (α)	Beta (β)	Selected Value	Pruned Node
Root (MAX)	$-\infty \rightarrow 3$	$+\infty$	3	None
Left MIN	$-\infty$	3	3	None
Right MIN	3	1	1	2

Final Answer

- Best Move for MAX: Choose the Left MIN subtree
- Final Minimax Value: 3
- Pruned Node(s): 2

Code

Minimax Algorithm with Alpha-Beta Pruning

```
import math
```

```
# Alpha-Beta Pruning function
```

```
def minimax(node, depth, alpha, beta, maximizingPlayer):
```

```
    # Leaf node reached
```

```
    if depth == 0:
```

```
        return node
```

```
    if maximizingPlayer:
```

```
        best_value = -math.inf
```

```
    for child in node:
```

```
        value = minimax(child, depth - 1, alpha, beta, False)
```

```
        best_value = max(best_value, value)
```

```
    alpha = max(alpha, best_value)
```

```
    # Alpha-Beta pruning
```

```
    if beta <= alpha:
```

```
        print("Pruned node:", child)
```

```
        break
```

```
    return best_value
```



```

else:
    best_value = math.inf

    for child in node:
        value = minimax(child, depth - 1, alpha, beta, True)
        best_value = min(best_value, value)

    beta = min(beta, best_value)

    # Alpha-Beta pruning
    if beta <= alpha:
        print("Pruned node:", child)
        break

    return best_value

```

```

# Game Tree
# Root MAX
tree = [
    [3, 5, 6], # Left MIN node
    [9, 1, 2]  # Right MIN node
]

```

```

# Initial values
alpha = -math.inf
beta = math.inf

```

```

# Run Minimax with Alpha-Beta Pruning
result = minimax(tree, 2, alpha, beta, True)

```

```
print("\nFinal Minimax Value:", result)
```

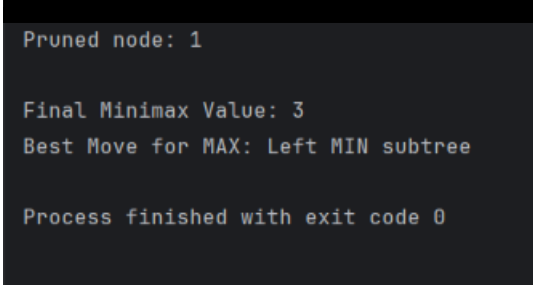
```
if result == min(tree[0]):
```

```
    print("Best Move for MAX: Left MIN subtree")
```

```
else:
```

```
    print("Best Move for MAX: Right MIN subtree")
```

Output

A terminal window with a dark background and light-colored text. The output shows the results of an Alpha-Beta Pruning search. The first line indicates that a node was pruned. The second line shows the final minimax value. The third line shows the best move for MAX. The fourth line shows the exit code.

```
Pruned node: 1

Final Minimax Value: 3
Best Move for MAX: Left MIN subtree

Process finished with exit code 0
```

Report

Alpha-Beta Pruning optimizes the Minimax algorithm by avoiding the evaluation of branches that cannot affect the final decision. During the left-to-right traversal, the left MIN node returns a value of 3, which updates the root's alpha value to 3. While evaluating the right MIN node, the beta value becomes 1 after visiting the second child, satisfying the pruning condition ($\beta \leq \alpha$). As a result, the remaining leaf node 2 is pruned without evaluation. The algorithm concludes that the left subtree is the best move for MAX, and the final minimax value is 3.